

PingFin – Banking Project

Team 13

Boutaarourte Aya
Maalmi Yamine
Tenjiti Imane
Vanschoenbeek Yelle

Tweede Bachelor Toegepaste Informatica

Integration Project 2
Sam Van Buggenhout
Jens Baetens

Inhoudstafel

1. Inleiding	3
1.1 Beschrijving van het project	3
1.2 Doel van het project	3
2. Schema van de opbouw van het project	3
2.1 Communicatie tussen GUI en API	3
2.2 Databankstructuur	4
3. Documentatie van de API	5
3.1 Beschrijving van API endpoints	5
3.2 Informatie over gebruikte methoden en parameters	5
3.3 Voorbeeldgebruik met Postman of vergelijkbare tool	6
4. Flow van een Payment Order	7
4.1 Flowchart die de stappen van een payment order doorheen het systeem illustreert	7
5. Beschrijving van het proces	8
5.1 Beschrijving plan van aanpak (uitgevoerde taken per dag , plan van aanpak, ...)	8
5.2 Werkverdeling en planning	10
5.3 Problemen en oplossingen	10
6. Handleiding voor het gebruik van de GUI	11
6.1 Functionaliteiten van de GUI	11
6.2 Gebruiksaanwijzingen (inclusief screenshots)	12
7. Reflectie	14
8. Extra bijlage	15

1. Inleiding

1.1 Beschrijving van het project

In dit project simuleren we een betalingssysteem waarbij verschillende banken met elkaar communiceren via een centrale clearing bank. Er zijn drie belangrijke partijen, we hebben de originator bank (OB), de clearing bank (CB) en de beneficiary bank (BB).

Wanneer een betaling wordt gestart bij de originator bank wordt deze niet rechtstreeks naar de andere bank gestuurd, maar eerst doorgestuurd naar de clearing bank. Deze dient als tussenpersoon en controleert/ valideert de betaling voordat deze eventueel verder wordt gestuurd naar de beneficiary bank.

Tijdens de workshop bouwen we zelf een applicatie die dit proces nadoet. Dit bevat het ontwerpen van een database, het ontwikkelen van een API voor communicatie tussen de banken en het bouwen van een eenvoudige gebruikersinterface. Daarnaast besteden we ook aandacht aan testing, foutafhandeling en logging, zodat het systeem betrouwbaar werkt.

1.2 Doel van het project

Het doel van dit project is om inzicht te krijgen in hoe een betalingssysteem werkt en hoe verschillende systemen met elkaar communiceren via API's. Daarnaast leren we hoe we een applicatie moeten opbouwen volgens een gestructureerde aanpak waarbij backend, database en frontend samenwerken.

Een ander belangrijk doel is het ontwikkelen van samenwerkingsvaardigheden. Door te werken met tools zoals GitHub en Trello leren we hoe we taken verdelen, voortgang opvolgen en efficiënt samenwerken binnen een team, zoals dat ook het geval later zal zijn in een professionele werkomgeving .

2. Schema van de opbouw van het project

Het project bestaat uit drie belangrijke onderdelen de GUI, de API en de databank. Deze onderdelen werken samen om betalingen correct te verwerken en weer te geven aan de gebruiker.

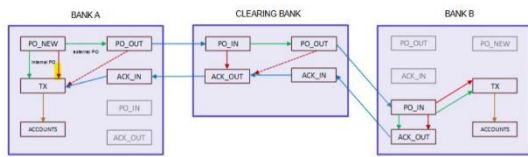
2.1 Communicatie tussen GUI en API

De GUI (gebruikersinterface) communiceert met de API via HTTP-requests. Wanneer een gebruiker een actie uitvoert zoals het bekijken of aanmaken van een betaling, stuurt de GUI een request naar de API.

De API verwerkt deze aanvraag, gaat de nodige logica uitvoeren en validaties en stuurt vervolgens een response terug naar de GUI. Op basis van deze response wordt de informatie weergegeven aan de gebruiker.

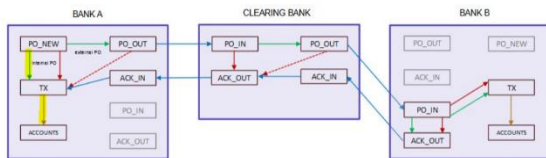
2.2 Databankstructuur

Use case 1:



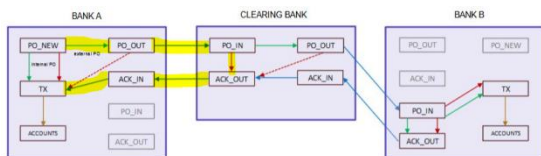
De betaling wordt gestopt bij de originator bank omdat de validatie faalt.

Use case 2:



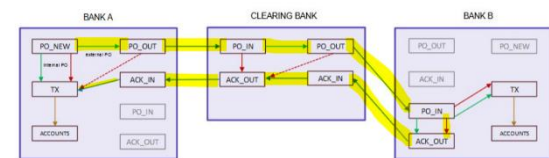
De betaling gebeurt intern binnen dezelfde bank en wordt rechtstreeks verwerkt.

Use case 3:



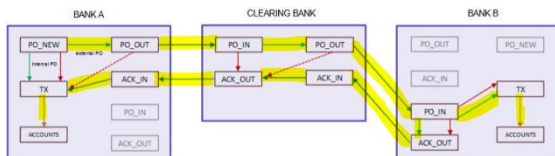
De betaling gaat naar de clearing bank, maar wordt daar afgekeurd en teruggestuurd.

Use case 4:



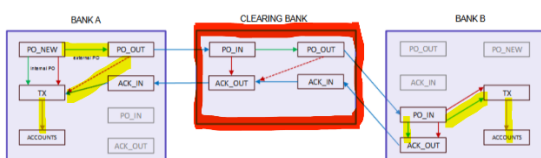
De betaling bereikt de beneficiary bank, maar faalt daar en wordt teruggestuurd.

Use case 5:



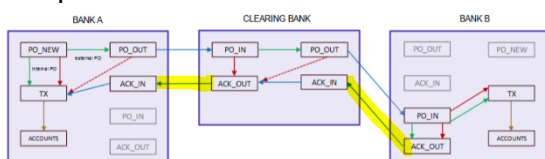
De betaling doorloopt alle stappen succesvol en wordt correct uitgevoerd.

Exception 1:



Als er geen antwoord komt van de clearing bank, wordt de aanvraag later opnieuw geprobeerd en gelogd.

Exception 2:

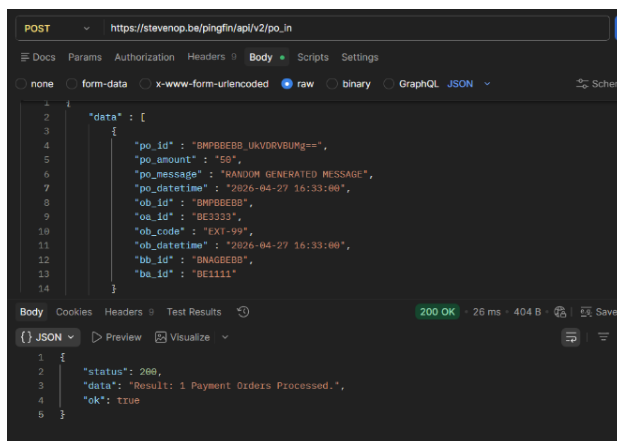


Berichten die niet worden opgehaald, blijven opgeslagen in de clearing bank tot ze verwerkt worden.

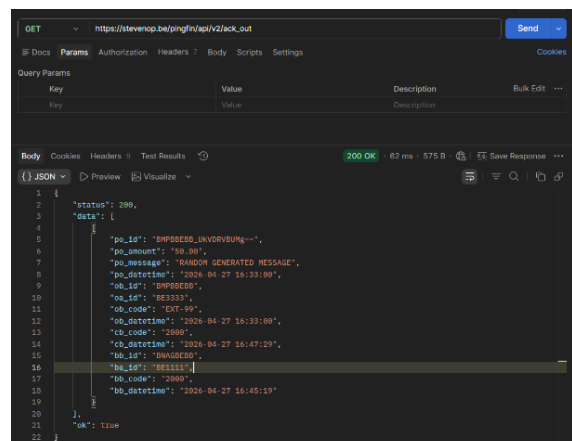
3.3 Voorbeeldgebruik met Postman of vergelijkbare tool

Voor het testen van onze API hebben we gebruik gemaakt van Postman. Hiermee konden we eenvoudig requests versturen naar de verschillende endpoints en de responses analyseren. Zo hebben we bijvoorbeeld een POST-request naar /po_in gestuurd om een nieuwe payment order te registreren. In de body hebben we alle nodige gegevens meegegeven, zoals het bedrag, de betrokken banken en de accounts. Als response kregen we een bevestiging dat de payment order succesvol verwerkt werd. Daarnaast hebben we GET-requests naar /ack_out gebruikt om acknowledgements op te halen. In de response konden we zien of de betaling succesvol was of dat er een fout was opgetreden. Zo zagen we bijvoorbeeld dat bij een fout in de clearing bank een specifieke errorcode werd teruggegeven en bepaalde velden, zoals bb_code, leeg bleven. We hebben ook verschillende scenario's getest, zoals een correcte betaling en een betaling met een fout (bv. een onbestaande bank). Op die manier konden we controleren of de validaties en foutafhandeling correct werkten.

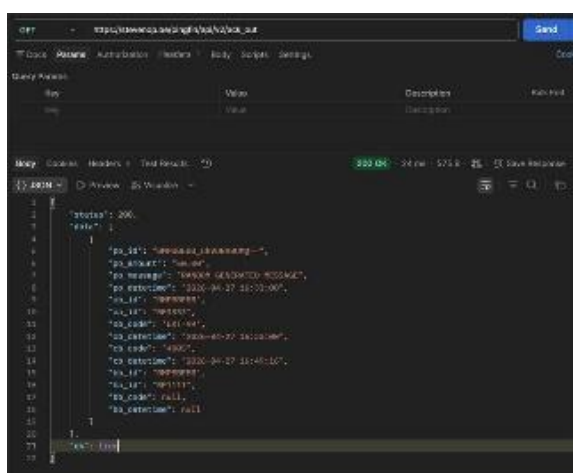
Dankzij al die testen in Postman kregen we een duidelijk beeld van hoe de API reageert in verschillende situaties en konden we bevestigen dat de communicatie tussen de verschillende onderdelen van het systeem correct verloopt.



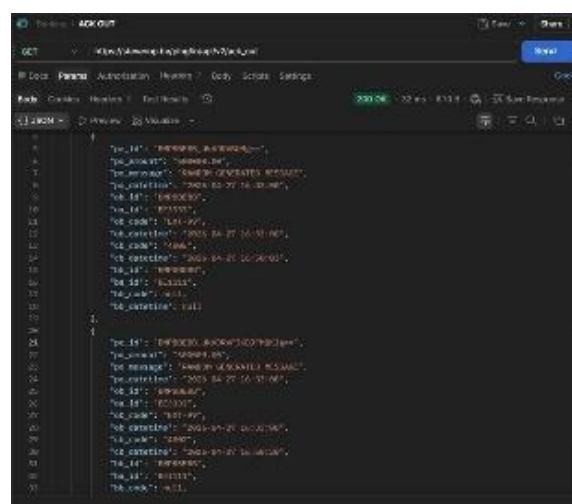
```
POST https://stevanop.be/pingfin/api/v2/po_in
{
  "data": {
    "po_id": "BMPBEBB_UKVOVBUM==",
    "po_amount": "50",
    "po_message": "RANDOM GENERATED MESSAGE",
    "po_datetime": "2026-04-27 16:33:00",
    "ob_id": "BMPBEBB",
    "oa_id": "BE3333",
    "ob_code": "EX1-99",
    "ob_datetime": "2026-04-27 16:33:00",
    "bb_id": "BNAGBEBB",
    "ba_id": "BE1111"
  }
}
{
  "status": 200,
  "data": {
    "Result": "1 Payment Orders Processed.",
    "ok": true
  }
}
```



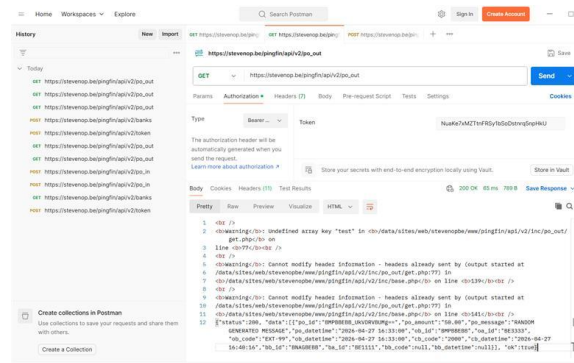
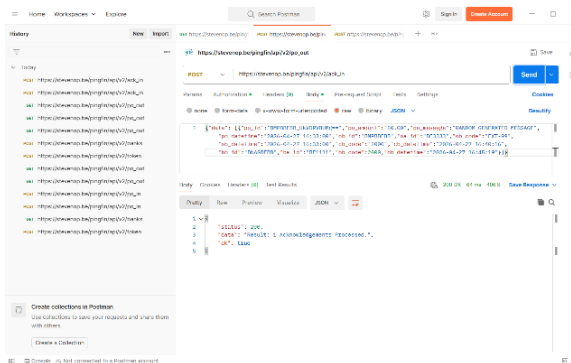
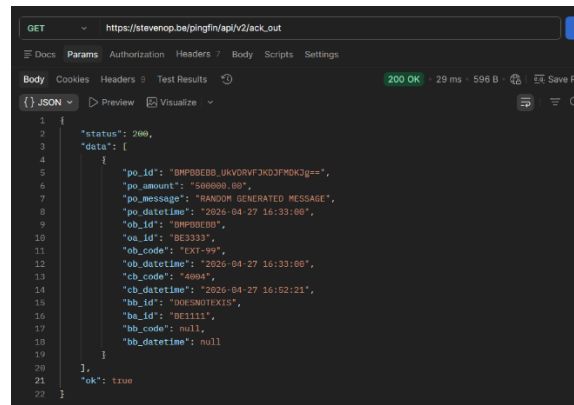
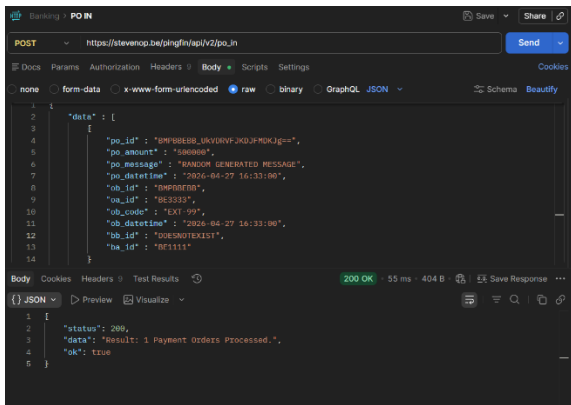
```
GET https://stevanop.be/pingfin/api/v2/ack_out
{
  "status": 200,
  "data": {
    "po_id": "BMPBEBB_UKVOVBUM==",
    "po_amount": "50.00",
    "po_message": "RANDOM GENERATED MESSAGE",
    "po_datetime": "2026-04-27 16:33:00",
    "ob_id": "BMPBEBB",
    "oa_id": "BE3333",
    "ob_code": "EX1-99",
    "ob_datetime": "2026-04-27 16:33:00",
    "cb_code": "2000",
    "cb_datetime": "2026-04-27 16:47:29",
    "ba_id": "BNAGBEBB",
    "bb_id": "BE1111",
    "bb_code": "2000",
    "bb_datetime": "2026-04-27 16:48:19"
  },
  "ok": true
}
```



```
GET https://stevanop.be/pingfin/api/v2/ack_out
{
  "status": 200,
  "data": {
    "po_id": "BMPBEBB_UKVOVBUM==",
    "po_amount": "50.00",
    "po_message": "RANDOM GENERATED MESSAGE",
    "po_datetime": "2026-04-27 16:33:00",
    "ob_id": "BMPBEBB",
    "oa_id": "BE3333",
    "ob_code": "EX1-99",
    "ob_datetime": "2026-04-27 16:33:00",
    "cb_code": "2000",
    "cb_datetime": "2026-04-27 16:48:19",
    "ba_id": "BNAGBEBB",
    "bb_id": "BE1111",
    "bb_code": "2000",
    "bb_datetime": "2026-04-27 16:48:19"
  },
  "ok": true
}
```

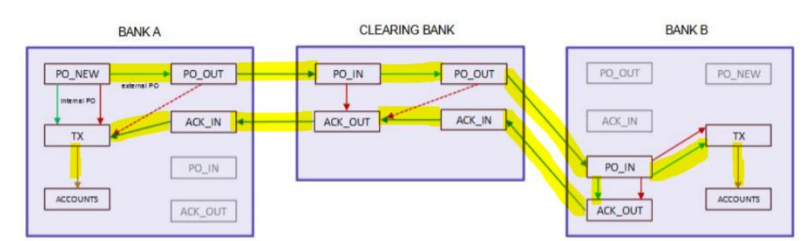


```
GET https://stevanop.be/pingfin/api/v2/ack_out
{
  "status": 200,
  "data": {
    "po_id": "BMPBEBB_UKVOVBUM==",
    "po_amount": "50.00",
    "po_message": "RANDOM GENERATED MESSAGE",
    "po_datetime": "2026-04-27 16:33:00",
    "ob_id": "BMPBEBB",
    "oa_id": "BE3333",
    "ob_code": "EX1-99",
    "ob_datetime": "2026-04-27 16:33:00",
    "cb_code": "2000",
    "cb_datetime": "2026-04-27 16:48:19",
    "ba_id": "BNAGBEBB",
    "bb_id": "BE1111",
    "bb_code": "2000",
    "bb_datetime": "2026-04-27 16:48:19"
  },
  "ok": true
}
```



4. Flow van een Payment Order

4.1 Flowchart die de stappen van een payment order doorheen het systeem illustreert



De flow van een betaling begint bij de originator bank (OB), waar de betaling wordt aangemaakt en gecontroleerd. Daarna wordt deze doorgestuurd naar de clearing bank (CB), die nog eens een extra controle zal uitvoeren. Als alles in orde is, gaat de betaling verder naar de beneficiary bank (BB), waar de transactie wordt uitgevoerd en het saldo van de accounts wordt aangepast.

Als er ergens in het proces een fout optreedt, wordt de betaling teruggestuurd naar de originator bank via een bevestiging (ACK).

5. Beschrijving van het proces

5.1 Beschrijving plan van aanpak (uitgevoerde taken **per dag**, plan van aanpak, ...)

- Maandag 27/04/2026

Tijdens de eerste dag hebben we ons vooral gefocust op de opstart en de analyse van het project. Om de communicatie vlot te laten verlopen, hebben we een Teams-groep aangemaakt waarin we alle nodige documenten kunnen delen. Voor de organisatie van het project hebben we een Trello-bord opgesteld, waarop we de taken verdelen en opvolgen. Zo kunnen we duidelijk zien welke taken nog moeten gebeuren, bezig zijn of al afgewerkt zijn.

Daarnaast hebben we zoals er werd gevraagd een GitHub-repository aangemaakt. Aya heeft alle teamleden toegevoegd zodat iedereen toegang heeft en we samen aan het project kunnen werken.

Nadat alle tools en communicatiekanalen waren opgezet, zijn we gestart met het analyseren van de *general messaging scheme*. We hebben de vijf use cases bestudeerd:

- use case 1: validatie faalt bij de originator bank
- use case 2: interne betaling binnen dezelfde bank
- use case 3: validatie faalt bij de clearing bank
- use case 4: validatie faalt bij de beneficiary bank
- use case 5: alle validaties slagen en de betaling succesvol is

Daarnaast hebben we ook de twee uitzonderingen (exceptions) besproken:

- exception 1: wat te doen als de originator of beneficiary bank geen antwoord krijgt van de clearing bank (bijvoorbeeld wanneer de API niet werkt)
- exception 2: wat er gebeurt met berichten in de clearing bank wanneer deze niet worden opgehaald

Iedereen heeft een use case geanalyseerd en deze aangeduid op de general messaging scheme door de trajecten visueel te markeren. Vervolgens hebben we onze uitleg met elkaar gedeeld en besproken zodat we tot één duidelijke uitleg.

De uitgewerkte trajecten hebben we daarna toegevoegd aan onze PowerPoint en zo onze analyse afgerond. Na de presentatie bij de leerkracht hebben we ook een naam gekozen voor onze bank namelijk RPH Bank.

Daarna zijn we gestart met het ontwerpen van de databankstructuur die we op dezelfde dag hebben afgewerkt. Tot slot hebben we mock-ups gemaakt om een beter visueel beeld te krijgen van de applicatie en zijn we begonnen met de documentatie van onze API met behulp van Docusaurus, een open-source static site generator.

- Dinsdag 28/04/2026

Vandaag hebben we verder gewerkt aan ons project en hebben we verschillende belangrijke onderdelen verder uitgewerkt en afgerond. We zijn de dag gestart met het ontwerpen van een logo voor onze bank, RPH Bank. Dit was belangrijk om ons project een eigen identiteit te geven en het geheel professioneler te laten ogen.

Daarna hebben we verder gewerkt aan de mockups van onze applicatie. We hebben een nieuwe, verbeterde versie gemaakt (versie 2), waarin we extra functionaliteiten hebben toegevoegd. In deze versie kan de gebruiker niet alleen betalingen invoeren, maar ook data bekijken zoals payment orders en members. Daarnaast hebben we knoppen toegevoegd om data te tonen of te verbergen, zodat de interface overzichtelijker blijft voor de gebruiker. Ondertussen werd ook de databankstructuur volledig afgerond. We hebben ervoor gezorgd dat alle nodige tabellen en relaties correct zijn opgesteld, zodat de gegevens van betalingen, accounts en acknowledgements goed kunnen worden opgeslagen en verwerkt. Dit vormt de basis voor de verdere werking van onze applicatie.

De documentatie van de API is vandaag ook volledig afgerond met behulp van Docusaurus. Tijdens dit proces zijn we echter een probleem tegengekomen, wanneer we de HTML-bestanden lokaal openden, werd de CSS niet correct geladen. Hierdoor zag de documentatie er niet goed uit. Om dit probleem op te lossen, hebben we beslist om onze documentatie te deployen via Vercel. Op die manier wordt alles correct weergegeven. Vervolgens hebben we de link naar deze online documentatie toegevoegd aan de readme van onze GitHub-repository.

Tot slot hebben we verder gewerkt aan het verslag. We hebben uitleg toegevoegd bij verschillende onderdelen en waar nodig screenshots ingevoegd om alles duidelijker te maken. Op die manier zorgen we ervoor dat het verslag verder wordt aangevuld.

-Woensdag 28/04/2026

Vandaag hebben we verder gewerkt aan het project en vooral gefocust op de API en de koppeling met de frontend. We zijn begonnen met het verder uitwerken van de API. Hierbij hebben we extra validaties toegevoegd zodat foutieve input beter wordt opgevangen (bijvoorbeeld verkeerde gegevens). Daarnaast hebben we ook een duidelijk overzicht gemaakt van de errorcodes zodat we beter kunnen zien welke fouten er kunnen optreden en wat de betekenis ervan is.

Verder hebben we ook gewerkt aan de UI. Er werd een login-functionaliteit aangemaakt zodat gebruikers zich kunnen aanmelden in de applicatie. Dit is belangrijk zodat enkel gebruikers met toegang tot de applicatie het kunnen gebruiken. Ondertussen hebben we ook de structuur van het project opnieuw herwerkt volgens de requirements van het netwerkteam zodat alles beter georganiseerd en overzichtelijk is voor beide teams.

We hebben ook een Dockerfile en een docker-compose file opgesteld, ook op aanvraag van het netwerkteam. Hiermee kunnen we de applicatie makkelijker opstarten en draaien. Ook hebben we verschillende routes aangepast binnen de API en verder gewerkt aan nieuwe endpoints. We hebben onder andere gewerkt aan de transacties endpoints en de log endpoints waarbij we functionaliteiten hebben toegevoegd om logs aan te maken en op te halen.

Daarnaast hebben we geprobeerd om de API te binden met de frontend. Hiervoor hebben we de JavaScript-structuur herschreven zodat deze beter aansluit op de API. We hebben ook een GET gedaan naar de clearing bank om bankgegevens op te halen.

We hebben ook alles in de juiste mappen gezet en getest. Tijdens het testen merkten we dat sommige endpoints (vooral de POST-endpoints) nog niet correct werken. Hierdoor lukt het momenteel nog niet om de API volledig te koppelen met de frontend. Omdat deze endpoints nog niet werken, kunnen we voorlopig niet verder en wachten we tot Yelle klaar is met het fixen van zijn endpoints.

Tot slot hebben we het project gepusht naar de branch api-feature/transactions zodat alle wijzigingen bewaard zijn.

- Donderdag 30/04/2026

Vandaag hebben we verder gewerkt aan het project en vooral gefocust op het afwerken en stabiliseren van de applicatie. We zijn begonnen met het verder fixen van de API. Hierbij hebben we verschillende fouten opgelost zodat alle endpoints correct werken. Uiteindelijk hebben we ervoor gezorgd dat alle endpoints volledig af zijn en doen wat ze moeten doen. Daarna hebben we gewerkt aan de frontend environment. We hebben dit volledig afgewerkt zodat de frontend correct kan communiceren met de API. Dit was nodig om alles goed te kunnen testen. Vervolgens hebben we de volledige applicatie getest. We hebben alle endpoints getest, zowel GET als POST, om zeker te zijn dat alles correct werkt. Hierbij hebben we ook extra aandacht gegeven aan de logs en de payments, omdat daar eerder nog problemen zaten. Deze hebben we gefixt en opnieuw getest. Daarnaast hebben we ook de site volledig afgewerkt. Dit wil zeggen dat de UI en functionaliteiten nu volledig klaar zijn en werken zoals verwacht. Verder zijn we gestart met het maken van de PowerPoint voor de eindpresentatie. We hebben al een eerste layout gemaakt en de structuur van de presentatie opgesteld. We hebben ook al enkele slides ingevuld.

Kort samengevat was vandaag vooral een afwerkdag waarbij we alles hebben gefixt, getest en klaargemaakt voor de presentatie.

5.2 Werkverdeling en planning

Binnen ons team hebben we de taken verdeeld op basis van ieders sterktes. Aya focuste zich vooral op validaties en backend, Imane op documentatie en frontend, Yamine op frontend en backend en Yelle op backend en de databank.

We hebben gebruik gemaakt van een Trello-bord om onze taken te plannen en op te volgen. Taken werden verdeeld in “to do”, “in progress” en “done” zodat iedereen duidelijk wist wat er moest gebeuren en wat er al afgewerkt was.

5.3 Problemen en oplossingen

Tijdens het project zijn we een probleem tegengekomen bij het gebruik van Docusaurus. De leerkracht vroeg om de documentatie in HTML-vorm, maar wanneer we de HTML-bestanden lokaal openden, werd de CSS niet correct geladen. Hierdoor zag de documentatie er niet goed uit en ontbrak de opmaak.

Om dit probleem op te lossen, hebben we ervoor gekozen om onze Docusaurus-website te deployen op Vercel. Op die manier wordt alles correct weergegeven inclusief de styling. We hebben vervolgens de link naar de online documentatie toegevoegd in de readme van onze GitHub-repository zodat de leerkracht de documentatie makkelijk kon bekijken.

Link naar de documentatie:

<https://rph-bank-dqf5ymhlm-usernayas-projects.vercel.app/v1/intro>

We hebben geprobeerd om de GUI in de juiste branch (feature branch) te plaatsen, maar we kwamen daarbij verschillende problemen tegen. Het lukte niet om de wijzigingen correct via

GitHub te pushen. Ook het wisselen tussen branches werkte niet goed, waardoor het werk per ongeluk in een andere branch terecht kwam.

Na wat zoeken bleek dat het probleem lag bij GitHub zelf. Uiteindelijk heeft hij GitHub geüpdatet en daarna werkte alles opnieuw zoals het moest.

Een ander probleem dat we zijn tegengekomen was bij CreateAccount (zie foto in bijlage). Het account werd wel correct aangemaakt, maar de balans werd niet ingevuld in de databank. Dit hebben we opgelost door de functies in de API aan te passen zodat de balance correct wordt meegestuurd en opgeslagen.

Daarnaast hebben we ook gemerkt dat het datumformaat niet correct was. De leerkracht had aangegeven dat dit zo niet goed was. Daarom hebben we overal hetzelfde formaat gebruikt door dit aan te passen in de API met:

```
DATE_FORMAT(po_datetime, '%Y-%m-%d %H:%i:%s') AS po_datetime
```

Op die manier worden alle datums correct weergegeven.

Een ander probleem was dat we de POST acknowledgment niet konden testen via

`/api/v1/acknowledgments/handle`.

Volgens ChatGPT lag dit aan een fout in het model en het ontbreken van een controller (req, res). Uiteindelijk hebben we gezien dat dit endpoint eigenlijk niet klopt volgens de documentatie. De juiste endpoints zijn:

`/api/v1/acks/in`

`/api/v1/acks/out`

We hebben dit dus aangepast zodat het overeenkomt met de API.

Verder werkte ook het aanmaken van een nieuwe payment (POST `/payments/new`) niet (zie foto in bijlage). We kregen hierbij een foutmelding (404). Volgens de analyse lag dit eraan dat de controller de JSON data niet correct mapped (bv. array wordt niet goed verwerkt). Ook hier hebben we de functies in de API aangepast zodat de data correct verwerkt wordt.

6. Handleiding voor het gebruik van de GUI

6.1 Functionaliteiten van de GUI

In versie 1 kan de gebruiker een betaling ingeven door het IBAN-nummer van de verzender, het bedrag en het IBAN-nummer van de ontvanger in te vullen. Daarnaast kan de gebruiker een betaling versturen of meerdere testbetalingen genereren.

In versie 2 is de interface uitgebreid met extra functionaliteiten. De gebruiker kan nu ook data bekijken zoals payment orders en members. Daarnaast zijn er knoppen voorzien om data te tonen of te verbergen, wat zorgt voor een beter overzicht.

6.2 Gebruiksaanwijzingen (inclusief screenshots)

Versie 1:

1. Open de applicatie.
2. Vul het IBAN-nummer van de verzender in.
3. Geef het bedrag in dat je wil versturen.
4. Vul het IBAN-nummer van de ontvanger in.
5. Klik op de knop om de betaling te versturen.
6. Optioneel : kan de gebruiker ook meerdere testbetalingen genereren door op de juiste knop te klikken.

SEPA Payments System

BANK

AMOUNT

BENEFICIARY IBAN

BUTTON

BUTTON

(MESSAGE)

Betaling versturen

VAN REKENING

BEDRAG (EUR)

10

IBAN BEGUNSTIGDE

Betaling sturen 10 willekeurige orders

Versie 2:

1. Open de applicatie.
2. Kies een account.
3. Vul het bedrag in.
4. Vul het IBAN-nummer van de ontvanger in.
5. Klik op "Send Payment" om de betaling te versturen.
6. Optioneel: Klik op "Send 10 Random Orders" om testbetalingen te genereren.
7. Gebruik de knoppen "Hide Data", "Hide New Payment Orders" of "Hide Members" om informatie te tonen of te verbergen.

SEPA Payments System

Choose your Account:

Enter amount (in EUR):

Enter beneficiary's IBAN:

Send Payment Send 10 Random Orders Hide Data Hide New Payment Orders Hide Members

Fetched Data

IBAN: be26579541237649 Balance: 500 EUR

New Payment Orders

id	amount	msg	date	ob_id	os_id	bib_id	ba_id
1	10	100	2014-01-01	0000	00000000	0000	00000000

Members

Name	BIC	Bank
Jonas Smil	CLIQBE33	MazdaBank

SEPA Payments System

CHOOSE YOUR ACCOUNT:

ENTER AMOUNT (IN EUR):

0.00

ENTER BENEFICIARY'S IBAN:

e.g. BE68 6390 0754 7034

Send Payment Send 10 Random Orders Hide Data Hide New Payment Orders Hide Members

Fetched Data —

New Payment Orders —

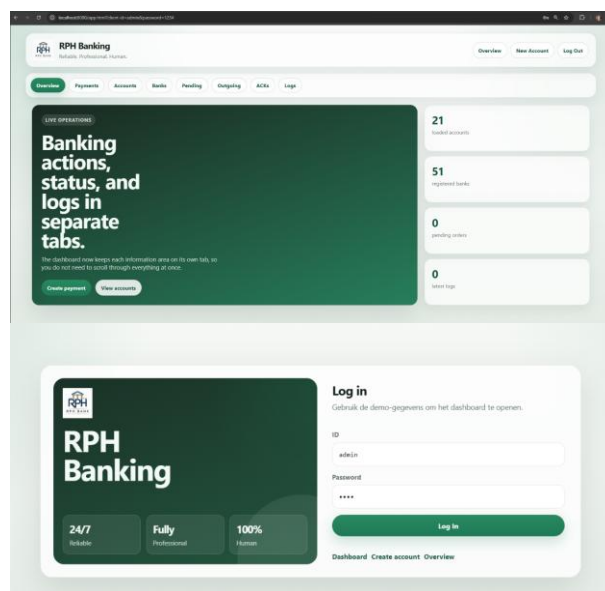
id amount msg date ob_id os_id bib_id ba_id

Members —

Name BIC Bank

Versie 3:

1. Open de applicatie en log in met de demo-gegevens.
2. Je komt terecht op het dashboard (Overview) waar je een overzicht ziet van accounts, banken, orders en logs.
3. Ga naar de tab "Payments" om een nieuwe betaling aan te maken.
4. Kies een from account uit de dropdown lijst.
5. Selecteer de beneficiary bank.
6. Vul het IBAN-nummer van de ontvanger in.
7. Vul het bedrag in (max €500).
8. Voeg een message toe (bijvoorbeeld "SEPA payment").
9. Klik op "Create Payment" om de betaling aan te maken.
10. Gebruik "Send Payments" om externe betalingen naar de clearing bank te sturen.
11. Gebruik "Handle Payments" om inkomende betalingen te verwerken.
12. Klik op "Refresh Banks" om de bankenlijst opnieuw op te halen.
13. Ga naar andere tabs zoals Accounts, Banks, Pending, Outgoing, ACKs en Logs om de status van het systeem te bekijken.
14. Gebruik "New Account" om een nieuwe rekening aan te maken.
15. Gebruik "Log Out" om uit te loggen.



BIC	NAME	MEMBERS
ARSPBE22	Argenta Spaarbank	TBD
AXABBE22	AXA Bank Belgium	Denzl De Mey, Oulkouh Touzani Rania, Seçken Arda, Ödemir Emirhan. Groep 2 - Central Bank. Lokaal HER 3-4405.
BARCBE88	Barclays Bank Ireland Plc Brussels Branch	Ilias, Brian, Saary, David
BBRUBEBB	Bankius	Amin, Oussama, Riyad, Tarik
BBVABEBB	Banco Bilbao Vizcaya Argentaria	TBD
BCOHBEBB	Banque Chaabi du Maroc	TBD
BCNKBEBB	NextPay	Semih Hakan Yildirim, Elias, Azzam
BIBLBE21	BestBank	Bunjamin, Amin, Sina
BKCHBEBB	Best Bank	Bunjamin, Amin, Sina
BKIDBE22	KDL OB	Yusuf, Johan, Youssef. Lokaal: 4407
BLUXBEBB	Banque de Luxembourg	TBD
BNEUBEB1	BNCE Euro Services	TBD

7. Reflectie

Tijdens het project hebben we veel geleerd, zowel technisch als op vlak van samenwerking. We hebben gewerkt met Node.js en Express voor het ontwikkelen van een API. Hierdoor hebben we beter inzicht gekregen in hoe een backend werkt en hoe je endpoints opbouwt. Daarnaast hebben we ook gewerkt met MySQL, waardoor we geleerd hebben hoe we data correct kunnen opslaan en ophalen uit een databank. We hebben ook ervaring opgedaan met softwareontwikkeling in het algemeen, waarbij we backend en database met elkaar moesten verbinden. Projectmanagement speelde ook een belangrijke rol. We hebben gewerkt met tools zoals Trello en Git om taken te verdelen en samen te werken binnen het team. Hierdoor hadden we een beter overzicht van wie wat moest doen en konden we efficiënter werken.

Verder hebben we geleerd hoe belangrijk API-ontwerp en validatie is. We hebben ervoor gezorgd dat foutieve input wordt opgevangen en dat de applicatie correct blijft werken. Ook hebben we veel getest met Postman, wat ons geholpen heeft om fouten sneller te vinden en op te lossen.

Daarnaast hebben we ook gewerkt met Docker. Dit heeft ons geleerd hoe we een geïsoleerde omgeving kunnen gebruiken om de applicatie te draaien en testen, zonder afhankelijk te zijn van lokale instellingen.

Naast wat goed ging, zijn er ook dingen die beter konden. We hebben gemerkt dat we onze

planning soms beter en grondiger hadden moeten maken. In het begin waren de requirements niet altijd even duidelijk, waardoor we later nog dingen moesten aanpassen. Ook hebben we het tijdsgebruik soms onderschat. Hierdoor ontstond er tijdsdruk op het einde van het project. In de toekomst zouden we meer gebruik maken van time-boxing, zodat we beter kunnen inschatten hoeveel tijd we aan een taak besteden.

Een ander belangrijk punt is dat we eerder hadden moeten beginnen met testen tijdens de ontwikkeling. Nu gebeurde dat vaak pas later, waardoor fouten pas laat ontdekt werden. Verder konden we ook beter werken aan een duidelijke en concrete taakverdeling, zodat iedereen exact weet wat zijn verantwoordelijkheid is.

Tijdens het project zijn we ook een aantal technische problemen tegengekomen. Zo hadden we problemen met Docusaurus, waarbij de HTML-documentatie lokaal niet correct werd weergegeven omdat de CSS niet geladen werd. Dit hebben we opgelost door de website te deployen op Vercel, zodat alles correct zichtbaar was. De link naar de documentatie hebben we toegevoegd in de README van GitHub. Ook ondervonden we problemen met GitHub branches. Het wisselen tussen branches werkte niet altijd correct en soms werd werk in de verkeerde branch gepusht. Uiteindelijk bleek dit probleem bij GitHub zelf te liggen en na een update werkte alles opnieuw zoals verwacht. Daarnaast hadden we een bug bij CreateAccount, waarbij de balans niet werd opgeslagen in de databank. Dit hebben we opgelost door de functies in de API aan te passen. We hebben ook het datumformaat moeten aanpassen, omdat dit niet correct was volgens de leerkracht. Dit hebben we opgelost door overal hetzelfde formaat te gebruiken in de API. Verder hadden we problemen met bepaalde endpoints, zoals acknowledgements en het aanmaken van nieuwe payments. Sommige endpoints klopten niet met de documentatie of werkten niet correct door fouten in de controller. Ook deze problemen hebben we opgelost door de API aan te passen.

Samengevat kunnen we zeggen dat we veel geleerd hebben uit dit project. Niet alleen op technisch vlak, maar ook op vlak van planning, samenwerking en probleemoplossend denken. Deze ervaring zal ons zeker helpen bij toekomstige projecten.

8. Extra bijlage

- logo



- Create account error

The screenshot displays a REST client interface with a POST request to `http://localhost:8080/api/v1/accounts`. The request body is a JSON object: `{ "balance": 50 }`. The response is a 201 Created status with a JSON body: `{ "id": "BE128528239849", "message": "Account successfully created." }`. To the right, a table shows account data with columns `id` and `balance`.

	id	balance
1	BE0000000000000000	670
2	BE03156643089505	0
3	BE1111111111111111	5,000
4	BE2222222222222222	1,250
5	BE3333333333333333	15,000
6	BE4444444444444444	320.5
7	BE5555555555555555	8,900
8	BE6666666666666666	100
9	BE66999317669520	0
10	BE7777777777777777	450
11	BE8888888888888888	10,500
12	BE9999999999999999	2,100

Below the REST client, a list of API endpoints for 'RPH Bank' is shown, with `POST CreateAccount` highlighted and marked with a green checkmark and a handwritten `1/2`.

- GET GetTransactions
- GET GetTransaction
- GET GetAllLogs
- GET GetAccounts
- POST CreateAccount 1/2
- GET GetOutgoingPayments
- GET GetIncomingPayments
- GET GetAcknowledgments

- Payment error

